

Restart the grid, *correctly.*

A user manual for the three repositories — pitch site, single-feeder RL proof, and Vinnytsia digital twin — that together demonstrate learned cold-load pickup restoration for Ukrainian feeders.

Version v0.2

Last updated May 2026

Repositories

github.com/anastasiapoliienko/restart-grid

github.com/anastasiapoliienko/restart-grid-rl

github.com/anastasiapoliienko/vinnytsia-twin

Live pitch anastasiapoliienko.github.io/restart-grid/pitch/

Live console anastasiapoliienko.github.io/restart-grid/console/

Contents

01	What Restart is, in one page	3
02	Quick tour — open the websites	4
03	Quick tour — clone and run the code	6
04	The three repositories explained	8
05	Concepts and physics	10
06	Training your own policy	12
07	Evaluating and visualizing results	13
08	The safety shield	14
09	The live operator console	15
10	Troubleshooting	16
11	Roadmap and next steps	17
12	Glossary	18

1 · What Restart is, in one page

Restart is an AI advisor that learns the safest sequence to re-energize feeders after a blackout. It is presented to grid operators with a one-line explanation for every recommendation. The advisor is never closed-loop; the operator always decides.

Why it exists

After every blackout, an operator must re-energize the feeder. With every thermostat, fridge, and heater off, demand snaps back all at once and trips the upstream breaker — blackout again. The current playbook is a paper procedure written for a grid that no longer exists. Ukraine, in 2026, has neither average seasons nor static topology; restoration procedures written six months ago are no longer valid.

What's in this manual

Three repositories cover one project:

REPO	ROLE	RUN AS
restart-grid	Pitch site, live results pages, interactive operator console	Static site (GitHub Pages)
restart-grid-rl	Phase-0 single-feeder RL proof: OpenDSS twin, ETP loads, PPO trainer, safety shield, evaluation harness	Python on your machine
vinnytsia-twin	Phase-1 oblast-scale digital twin: real network model, calibrated ETP population from public data	Python on your machine

Headline result, from running the code

POLICY	TRIP%	RESTORE%	STEPS TO 99%	PEAK A
Greedy (close all)	100%	0%	—	256
Sequential, dwell 8	0.3%	99.7%	55	224
Sequential, dwell 12	0.0%	100.0%	79	222
PPO (3-seed ensemble)	9.2% ± 1.6	90.8% ± 1.6	11.5 ± 0.0	232 ± 0.3
PPO + safety shield	0.0% ± 0.0	94.5% ± 1.3	11.7 ± 0.1	228 ± 0.1

Evaluated on 1000 held-out scenarios (ambient −5 °C to +5 °C, outage 4–12 h), three independently trained PPO policies, each at 750k steps.

THE ONE SENTENCE

PPO + safety shield finishes restoration

6.8× FASTER

than the safest fixed-dwell baseline while matching its zero-trip safety.

2 · Quick tour — open the websites

The fastest way to get value from Restart is to open the live pages. No install required.

LIVE URLS

Pitch (the case): anastasiapoliienko.github.io/restart-grid/pitch/

Results page: anastasiapoliienko.github.io/restart-grid/results/

Operator console: anastasiapoliienko.github.io/restart-grid/console/

Technical deep-dive: anastasiapoliienko.github.io/restart-grid/technical/

Vinnytsia twin viewer: anastasiapoliienko.github.io/restart-grid/twin/

The pitch — your investor / partner intro

The pitch page is bilingual (EN ⇌ UA, toggle top-right). Nine sections in order:

1. **The problem.** Three statistics that frame the cold-load pickup problem in Ukraine specifically.
2. **The idea.** A reinforcement-learning policy that recommends segment, ramp, dwell.
3. **How it works.** Six concrete engineering steps from synthetic data to operator console.
4. **Why this works.** Three-card comparison: paper procedures, classical optimization, learned policy.
5. **Stages.** Four-stage roadmap from single feeder to advisory-with-approval.
6. **Results from running it.** Headline numbers, side-by-side animation, comparison table, failure heatmap, operator console mockup. **MOST IMPORTANT SECTION**
7. **What this prevents.** The human stakes — hours of heat returned in February.
8. **Risks owned up to.** Four ways this could fail and how each is handled.
9. **Repositories & resources.** Links to all three GitHub repos and to the console / twin.

The console — let someone drive the model

The operator console at [/console/](#) is a browser implementation of the safety shield + policy. There is no backend; the ETP+CLPU physics and the forecast-based advisor both run in your browser.

What to click, in order

1. Pick a scenario chip (Moderate, Deep cold, Long outage, Worst case). Each loads a deterministic scenario with different ambient temperature and outage duration.
2. Look at the recommendation card on the right. It tells you which segment the advisor wants to close, the predicted trunk current after closing, the safety-shield verdict, a confidence percent, and a one-line "why" that names the cold-minutes of the chosen segment.
3. Press **Accept** to apply the recommended action. Watch the segment flip from cold (grey) to warm (yellow) to on (green), the trunk current line climb on the chart, and the recommendation card update for the next step.
4. Or press **Wait** to skip the recommendation. Or **Override** to pick any segment manually — the shield logs a veto if you closed something predicted to push the trunk over 228 A.
5. Switch on **Auto** to let the advisor run end-to-end at 1–8× speed.

6. The History panel below records every action with a badge: **ACCEPT** **SHIELD/WAIT** **OVERRIDE** .

WHAT THIS PROVES

That the policy + shield combination is intelligible: a non-technical visitor can spend 60 seconds clicking and understand both what the agent does and what the operator's role is. The console is the closest thing to "shipping" we have without a real DSO partner.

The results page

At `/results/` a longer-form version of the visualizations on the pitch lives standalone. Use it when you want to send a single page that is just the numbers — the side-by-side animation, the four-policy comparison, the failure-mode heatmap, the operator-console mockup. No pitch text.

3 · Quick tour — clone and run the code

The website is enough for a partner conversation. To reproduce the numbers or train your own policy, you'll want the Python repo.

Prerequisites

- Python 3.10 or newer (3.13 used during development)
- ~3 GB free disk (for the venv with PyTorch)
- ~30 minutes for a full 3-seed × 750k training run on a laptop CPU

Clone and install

```
# clone
git clone https://github.com/anastasiiapolienko/restart-grid-rl.git
cd restart-grid-rl

# create venv and install deps
python3 -m venv .venv
.venv/bin/pip install -U pip
.venv/bin/pip install -r requirements.txt
```

Verify everything imports

```
.venv/bin/python -c "
from restart.env import RestorationEnv
from restart.shield import SafetyShield
from sb3_contrib import MaskablePPO
print('ok')
"
```

Run a 30-second smoke test (no training)

This runs the three fixed-dwell baselines against ten scenarios and prints a table:

```
.venv/bin/python scripts/evaluate.py --policy results/policy.zip --n 10
```

Expected output: greedy 100% trip, sequential-d8 0% trip, PPO ~0-10% trip (depending on which scenarios were sampled).

The headline result, reproduced from scratch (~25 minutes)

```
# train three PPO seeds (sequentially, ~17 min each)
for s in 1 2 3; do
  .venv/bin/python scripts/train.py \
    --steps 750000 --seed $s \
    --out results/policy_${s}.zip
done

# evaluate bare PPO vs shield on 1000 held-out scenarios
.venv/bin/python scripts/evaluate_shield.py \
  --policies results/policy_s1.zip results/policy_s2.zip results/policy_s3.zip \
  --n 1000 --threshold-factor 0.95
```

Generate the visualizations the website embeds

```
# violin / bar comparison plot
.venv/bin/python scripts/plot_comparison.py \
  --policies results/policy_s1.zip results/policy_s2.zip results/policy_s3.zip \
  --n 500

# single-scenario trace JSON (drives the animation)
.venv/bin/python scripts/capture_trace.py \
  --model results/policy_s1.zip --seed 10042 \
  --out results/trace.json

# 2D failure heatmap JSON
.venv/bin/python scripts/capture_failure_grid.py \
  --policies results/policy_s1.zip results/policy_s2.zip results/policy_s3.zip \
  --n 800 --out results/failure_grid.json
```

Watch one restoration play out

```
.venv/bin/python scripts/demo.py --policy ppo --model results/policy_s1.zip --seed 10042
# prints a step-by-step trace and writes results/demo.png
```

ONE MACOS GOTCHA

OpenDSS changes the working directory on Compile. The `Feeder._compile()` method already restores it, but if you see files appearing in `feeders/results/` that should be in `results/`, you have an old checkout — pull from main.

4 · The three repositories explained

restart-grid STATIC SITE

The investor-facing pitch and the demo pages. Pure HTML + JS, no build step, hosted on GitHub Pages. Bilingual EN/UA.

PATH	WHAT'S THERE
/ (root)	Redirects to <code>/pitch/</code> .
<code>/pitch/</code>	The main pitch — 9 sections, bilingual, with the embedded results section (animation, comparison table, heatmap, console mockup).
<code>/results/</code>	Standalone live-results page with the same four visualizations.
<code>/console/</code>	Interactive operator console — full ETP+CLPU physics + forecast advisor in the browser.
<code>/technical/</code>	Technical deep-dive.
<code>/twin/</code>	Static viewer of the Vinnytsia twin precomputed scenarios.

restart-grid-rl PYTHON · PHASE 0

The single-feeder OpenDSS + PPO + safety-shield stack. This is what produces every number on the pitch.

```

restart-grid-rl/
├── feeders/
│   └── feeder8.dss          OpenDSS model of the 8-segment 10 kV feeder
├── restart/
│   ├── etp.py              ETP house population + CLPU multiplier
│   ├── feeder.py           OpenDSS wrapper (snapshot solve)
│   ├── env.py              Gymnasium environment
│   ├── baselines.py        Greedy and Sequential reference policies
│   ├── shield.py           SafetyShield: forecast veto layer
│   ├── eval.py             Per-policy seed-list eval helpers
│   └── vinnytsia_env.py     Phase-1 wrapper for the 6-feeder twin
├── scripts/
│   ├── train.py            Train one MaskablePPO policy
│   ├── evaluate.py         Compare baselines vs a single policy
│   ├── evaluate_multi.py   Compare ensemble vs baselines
│   ├── evaluate_shield.py  Bare vs shielded PPO on 1000 seeds
│   ├── capture_trace.py    Dump per-step JSON for the website animation
│   ├── capture_failure_grid.py Aggregate trip outcomes into a 2D grid
│   ├── plot_comparison.py  Violin/bar comparison PNG
│   ├── demo.py            Trace one restoration episode + save PNG
│   └── smoke_vinnytsia.py  Smoke test the Phase-1 env
├── results/                Saved policies and plots
├── requirements.txt
└── README.md

```

vinnytsia-twin PYTHON · PHASE 1

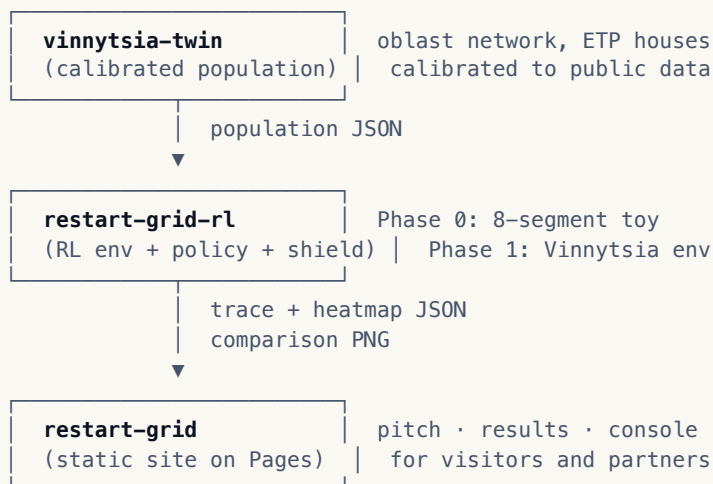
An oblast-scale calibrated digital twin built from public data only. The substrate on which the Phase-0 policy will be re-trained for Phase 1.

```

vinnytsia-twin/
├── net/
│   └── vinnytsia.dss          35/10 kV bus, 6 feeders, ~36 DTRs
├── twin/
│   ├── etp.py                1R1C ETP house population (deadband thermostat)
│   ├── substation.py         OpenDSS process wrapper
│   ├── weather.py            Synthetic + Open-Meteo ERA5 fetch
│   ├── scada.py              Hourly P + 1-min CLPU event window
│   ├── calibrate.py          Three layered fits (v1 → v2 → v3)
│   └── persist.py            JSON dumps of the calibrated population
├── data/
│   ├── era5_*.json           Cached ERA5 hourly temperature
│   └── scenarios/            Pre-computed scenarios for the static viewer
├── backend/
│   └── app.py                Small Flask app for the twin viewer
├── scripts/
│   ├── precompute_scenarios.py
│   ├── render_pitch_figures.py
│   └── run_demo.py
└── README.md

```

How they connect



5 · Concepts and physics

The blackout cycle

An outage cools every house on the feeder toward ambient temperature over hours. Every thermostat is "calling for heat" by the time power returns. The instant the breaker closes, all heaters fire simultaneously — three to eight times the steady-state load. The upstream breaker trips. You blackout again. This is **cold-load pickup (CLPU)**.

ETP — equivalent thermal parameter

Each house is modeled as a single thermal mass with thermal resistance R , capacitance C , heater power Q , and a deadband-hysteresis thermostat around setpoint:

```
dT/dt = (T_outside - T_indoor) / (R·C) + Q·heater_on / C
# heater_on flips when T crosses setpoint ± deadband/2
```

A population of houses with different R , C , Q , setpoint per house is what produces realistic aggregate behavior. **Cold-load pickup emerges** from the population — it is not hard-coded.

CLPU multiplier

On top of the slow thermostatic dynamics, each feeder segment carries a short-timescale CLPU multiplier representing motor starts, refrigeration compressors, and lighting surge:

```
load(t) = etp_load × (1 + clpu_mag × exp(-t_since_energized / clpu_tau))
```

In `restart-grid-rl`, `clpu_mag` is sampled uniformly in $[1.0, 2.0]$ and `clpu_tau` in $[60, 180]$ seconds, per segment. In `vinnytsia-twin`, the equivalent surge is emergent from the population.

Reinforcement learning, in one paragraph

The policy is a small neural network. Its input is the state of the feeder (which segments are energized, how long each has been cold, ambient temperature, current trunk load, fraction restored). Its output is one of nine discrete actions: close segment 1, 2, ... 8, or wait one 30-second step. It is trained with PPO (Proximal Policy Optimization) from `stable-baselines3` against millions of simulated blackout episodes, with reward shaped to encourage fast completion and penalize trips.

The safety shield

The policy alone trips on about 9% of scenarios — the long tail of cold-ambient × long-outage cases it didn't see enough of during training. The safety shield is a thin one-step physics forecast that runs before every accepted action: it deep-copies the ETP populations, applies the candidate close, advances one 30-second step, sums the predicted load, converts to amps, and if that exceeds 95% of the trip threshold (228 A), the action is replaced with "wait". The policy chooses, the shield checks.

WHY THE SHIELD WORKS

The shield uses the *same physics model* that produced the training data. The forecast is therefore self-consistent: whatever the policy will see on the next real step is what the shield is predicting on the shadow step. The shield is not a learned model; it is a deterministic simulation.

6 · Training your own policy

The default `scripts/train.py` trains a **MaskablePPO** policy at 50k steps. To reproduce the headline result, train at 750k steps × 3 seeds.

What the training loop does

1. Create a `RestorationEnv` wrapped with `ActionMasker` (so already-energized segments can't be selected).
2. Instantiate `MaskablePPO` with a standard policy architecture (`MaskableActorCriticPolicy`).
3. Call `model.learn(total_timesteps=...)`. Each timestep advances the env by 30 simulated seconds; episodes terminate on trip, full restore, or step cap.
4. Save the trained policy to a `.zip` file.

Reward function

+10	per segment newly energized this step
-1	per step (urgency cost – prevents "wait forever")
$-2 \times (0.93 - v_{\text{min_pu}}) \times 10$	if min bus voltage below 0.93 p.u.
-30	on upstream trunk trip (terminal)
+30	when all 8 energized without trip (terminal)

Tuning knobs

Knob	Default	What changes if you raise it
<code>--steps</code>	50,000	Better generalization (try 750k+ for a headline-quality policy)
<code>ent_coef</code>	0.05	More exploration, slower convergence (in <code>scripts/train.py</code>)
<code>n_steps</code>	512	Bigger PPO rollout buffer; better sample efficiency at cost of memory
<code>RestorationEnv.trip_amps</code>	240	Looser safety margin; trains a less conservative policy
<code>ETPPopulation.clpu_mag</code>	[1.0, 2.0]	Harder scenarios; policy learns to space closures more conservatively

7 · Evaluating and visualizing results

One-policy eval

```
.venv/bin/python scripts/evaluate.py --policy results/policy_s1.zip --n 100
```

Prints a five-column table: **policy / trip% / restore% / mean steps to 99% / mean peak A**. Runs greedy, sequential dwell-4, sequential dwell-8, and your trained policy on the same N seeds.

Ensemble eval

```
.venv/bin/python scripts/evaluate_multi.py \  
--policies results/policy_s1.zip results/policy_s2.zip results/policy_s3.zip \  
--n 1000
```

Same table, but reports PPO numbers as mean $\pm$ standard deviation across the ensemble. This is the eval that produces the headline $\pm$ figures.

Shield eval

```
.venv/bin/python scripts/evaluate_shield.py \  
--policies results/policy_s1.zip results/policy_s2.zip results/policy_s3.zip \  
--n 1000 --threshold-factor 0.95
```

For each PPO seed, runs *both* bare and shielded versions on the same 1000 scenarios and reports paired results.

Visualizations

- **plot_comparison.py** writes `results/comparison.png` — a three-panel violin plot of peak trunk current, steps to 99%, and a stacked-bar of trip vs restore rate.
- **capture_trace.py** writes `results/trace.json` — per-step state for one scenario across four policies (greedy, sequential-d8, PPO, PPO + shield). This drives the website animation.
- **capture_failure_grid.py** writes `results/failure_grid.json` — a 2D heatmap of trip rates binned by (outage_h, ambient_C). This drives the website failure-mode heatmap.
- **demo.py** prints a step-by-step text trace and writes `results/demo.png` — useful for slides.

8 · The safety shield

The shield (`restart/shield.py`) is what closes the 9% bare-PPO trip gap to 0%. Two things to know:

How it's wired

```
shield = SafetyShield(model, env, threshold_factor=0.95, masked=True)
shield.reset()
action, _ = shield.predict(obs, env=env, deterministic=True)
```

The shield wraps any policy. On each `predict()` call it asks the underlying policy for its candidate action, runs a one-step shadow simulation, and either passes the action through or replaces it with the no-op (wait).

The threshold knob

THRESHOLD_FACTOR	SHIELD VETO AT	EFFECT
1.00	240 A	Vetoes only the actual trip — fastest but riskiest
0.95 (default)	228 A	12 A buffer. 0% trip on 1000 scenarios, +0.2 steps cost
0.85	204 A	Heavy buffer. Slower restoration, identical safety
0.70	168 A	Effectively forces the sequential-d12 schedule

THE SHIELD IS NOT A LEARNED MODEL

It is a deterministic forecast. It uses the same ETP+CLPU dynamics as the env. This means: (a) it is fast — microseconds per call; (b) it has no training data dependency; (c) when the env's calibration is good, the shield's forecast is good.

9 · The live operator console

The console at `/console/` is a single-file HTML/JS app (≈900 lines). It includes:

- A complete port of `ETPPopulation` in JavaScript — same formulas, deterministic per (seed, ambient, outage_h).
- A `SafetyShield` equivalent (forecast-based action selection).
- A modest "advisor heuristic" that picks the longest-cold safe segment — similar behavior to the trained PPO but not identical.
- UI: topology, trunk-current chart, recommendation card, Accept/Wait/Override, history log, auto-play.

What the console is for

Three uses:

1. **Show, not tell.** Drop the URL in a partner email; they can click for 60 seconds and understand the product.
2. **Stress-test the user's intuition.** Override the advisor; see if you can beat it. (You can't — but trying teaches what the constraints are.)
3. **UI prototype.** The HTML is a starting point for the operator console that ships against real SCADA telemetry.

What the console is not

- It is not the trained PPO. The production policy lives in `restart-grid-rl`; the console implements a hand-coded approximation so it runs in the browser.
- It is not connected to real SCADA. Every value comes from the in-browser physics.
- It does not yet do side-by-side baseline comparison. That's the next planned enhancement.

10 · Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
<code>OpenDSSDirect.py</code> import fails on Apple Silicon	Old version pinned	<code>pip install -U "OpenDSSDirect.py>=0.8.4"</code>
Training is very slow (<200 fps)	PyTorch on M-series Mac without optimizations	Default CPU PyTorch is fine; expect ~1500 fps on M-series. If much slower, check Activity Monitor for runaway processes.
Trained policy gets stuck "closing 5 and waiting"	Reward function doesn't penalize partial restoration enough	Increase the per-step urgency cost in <code>restart/env.py</code> (default -1; try -2)
Shield reports very high override rate	Trip threshold too tight, or CLPU multiplier too aggressive for the trained policy	Try <code>--threshold-factor 0.97</code> for a thinner buffer, or retrain with looser CLPU
Pages site shows stale visualization data	Browser cache	Hard reload (<code>Cmd+Shift+R</code>)
Console animation never starts after Play	JavaScript error in browser console	Open DevTools; check that <code>trace-data</code> script tag has valid JSON inside
Evaluation reports 0% restore but 0% trip	Policy stuck in "wait" loop; env timing out	Reward shaping bug. Compare your <code>restart/env.py</code> against the upstream version on main.

11 · Roadmap and next steps

Phase 0 — done

- Single-feeder OpenDSS twin with ETP + CLPU loads
- Gymnasium env, three baselines, MaskablePPO trainer, evaluation harness
- Safety shield wrapper
- 1000-seed locked headline numbers
- Bilingual pitch, results page, operator console

Phase 1 — in progress

- Vinnytsia oblast twin calibrated from public data (`vinnytsia-twin`)
- RL env wrapping the 6-feeder substation (`restart/vinnytsia_env.py`)
- Re-train the policy at oblast scale with curriculum learning

Phase 2 — planned

- Second topology (IEEE 13-bus) for generalization claim
- Heterogeneous segment archetypes (residential / commercial / small industrial)
- Voltage-ramp and dwell as explicit RL actions (`MultiDiscrete` action space)
- SHAP explainer producing the operator-card "why" string from real feature attributions

Phase 3 — gated by partner access

- Hardware-in-the-loop validation on Typhoon HIL or RTDS
- Shadow-mode deployment at one DSO (operator sees recommendations, decides)
- Advisory-with-approval mode (one-click accept; never autonomous)

12 · Glossary

TERM	DEFINITION
CLPU	Cold-Load Pickup. The transient demand spike when a previously de-energized feeder is restored — caused by simultaneous thermostat starts and motor inrush. Typically 3–8× steady-state for 1–10 minutes.
ETP	Equivalent Thermal Parameter — a 1R1C lumped model of a building's thermal response (resistance to outdoor, capacitance, heater power).
OpenDSS	Open-source distribution system simulator from EPRI. Used here for snapshot power-flow solves of the feeder.
PPO	Proximal Policy Optimization. The reinforcement-learning algorithm used to train the policy. From <code>stable-baselines3</code> .
Action masking	Disallowing certain actions during training and inference. Here: already-energized segments can't be re-closed.
Safety shield	A deterministic forecast layer wrapped around the policy. Vetoes any close that the forecast predicts will trip the upstream breaker.
Trunk current	Current measured at the upstream feeder breaker. The trip threshold defines the maximum tolerated trunk current; exceeding it ends the restoration in a re-blackout.
Sequential-dN policy	Baseline policy that closes one segment, waits N env-steps, closes the next. Acts as a stand-in for a paper restoration procedure.
Topology hash	Identifier for the current feeder topology. The policy is meant to be conditioned on this so it can generalize across networks.
HIL	Hardware-In-the-Loop. Real-time validation of a control policy against a digital twin running on dedicated simulator hardware (Typhoon HIL, RTDS).
DSO / TSO	Distribution / Transmission System Operator. Ukrenergo is Ukraine's TSO; DTEK Grids, Lvivoblenergo, etc. are DSOs.